Introduction to High Performance Computing

UNIVERSITY OF CAMBRIDGE

Bioinformatics Training

“High Performance Computing most generally refers to the practice of aggregating computing power in a way that delivers much higher performance than one could get out of a typical desktop computer or workstation in order to solve large problems in science, engineering, or business.”

HPC vs. local computer

- Shared resources
- Administrative rights
- Parallel computing
- Specific nodes for different resource needs
- Reliability, redundancy, maintenance, safety

- Cost of running
- Impact on the environment

Parallel computing

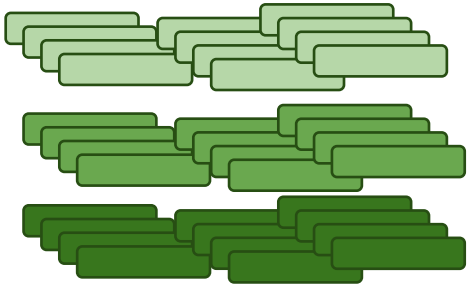
- Accelerate the run by using **multiple CPU cores** for calculation
  - Multiple CPUs are common in standard desktops/laptops or even smartphones
  - But HPC clusters have higher CPU count per computer (e.g. up to 112 CPUs at Cambridge)
- Run **multiple tasks in parallel** (e.g. run samples through an analysis pipeline)
  - Without HPC you would need multiple desktops or use some form of distributed computing (e.g. SETI@home, Folding@home)
- Perform a single task on multiple computers (nodes)
  - Although not as common, this can only be achieved on a HPC cluster

Topology of the HPC

Login nodes

Queue manager / Job Scheduler

Production/Compute nodes

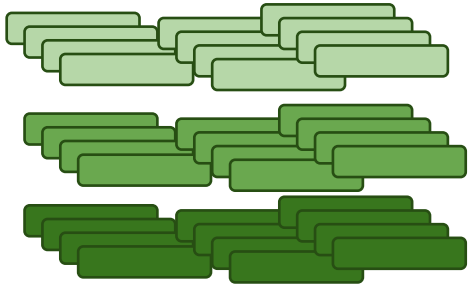


Topology of the HPC

Login nodes

Queue manager / Job Scheduler

Production/Compute nodes

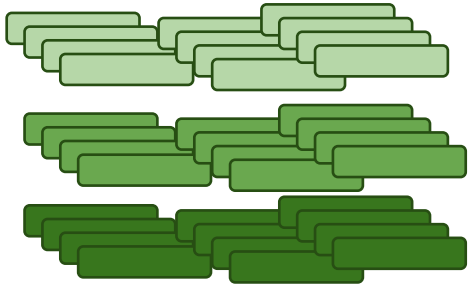


Topology of the HPC

Login nodes

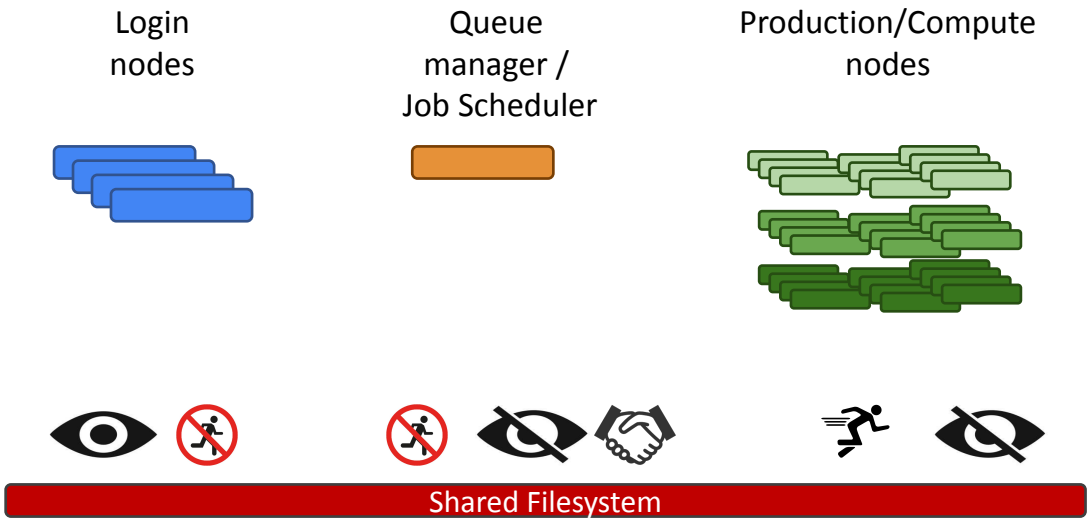
Queue manager / Job Scheduler

Production/Compute nodes



Shared Filesystem

Topology of the HPC



Vs.

Login nodes

- SSH login, **no GUI**
- SFTP / SCP file transfer
- Running your scripts on a login node is very tempting, but **you must resist!**

- Learn the way of direct file transfer solutions (wget, rsync, specific API, …)
- Use commands **ethically!**

Queue manager / Job Scheduler

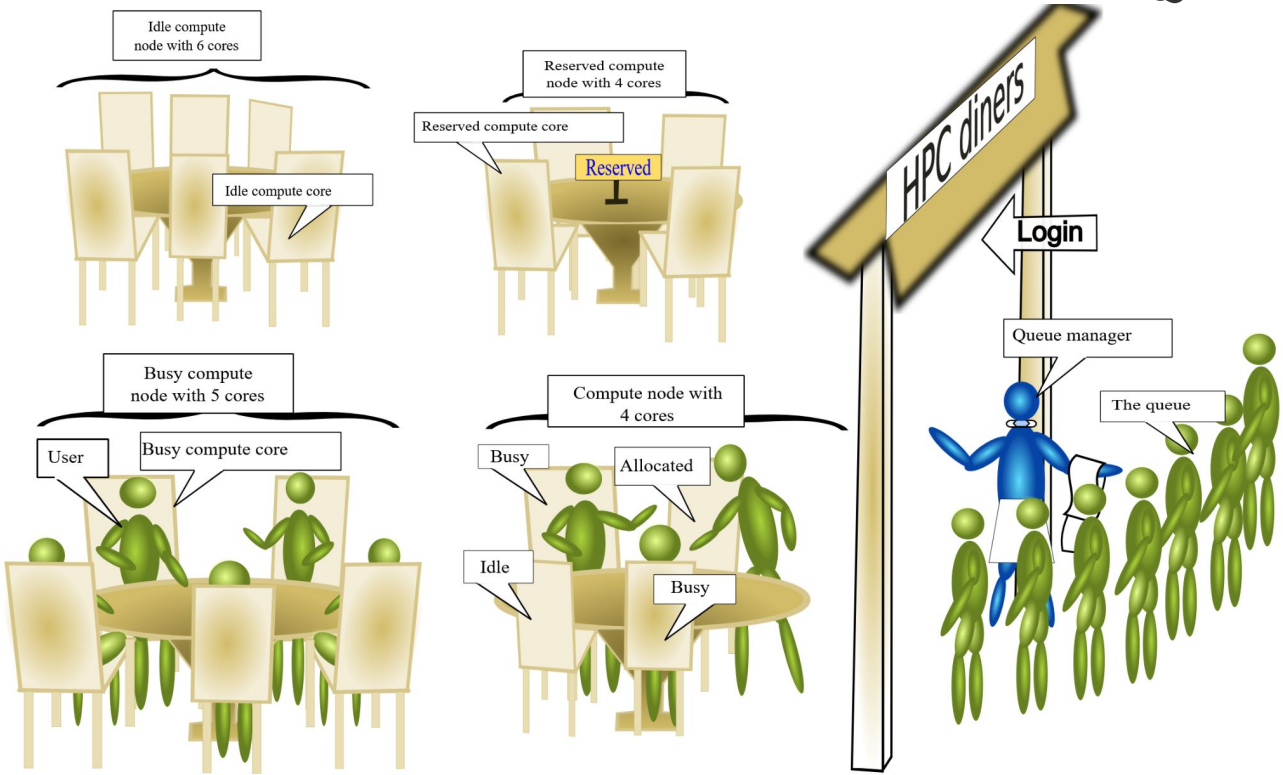


image source

Queue manager / Job Scheduler

Single computer environment (interactive use):

```
$ bowtie2 –x ref_index -1 reads_1.fastq -2 reads_2.fastq
```

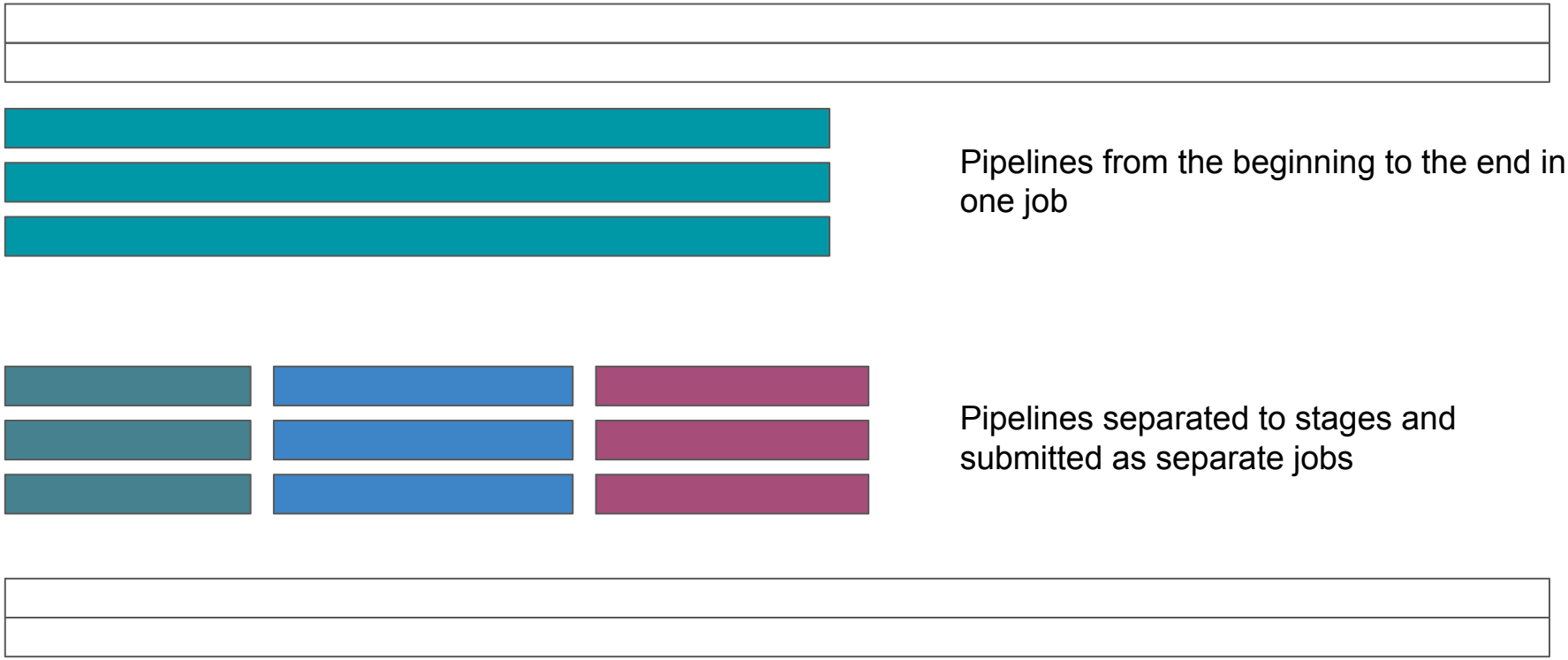
HPC (using scheduler):

```
$ sbatch RESOURCE_REQUEST mapping.sh
$ squeue
  JOBID PARTITION  NAME     USER   ST   TIME  NODES NODELIST
  2048  highmem    mapping  ht123  R    0:02  1     highmem-node01
```

Spend time thinking about your resource requests

- Reduce time and number of CPU cores if you can (backfilling)
- Monitor production node usage
- Time and CPU core restrictions

Work together with the Job Scheduler



Pipelines from the beginning to the end in one job

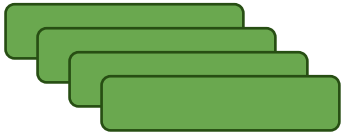
Pipelines separated to stages and submitted as separate jobs

Work together with the Job Scheduler

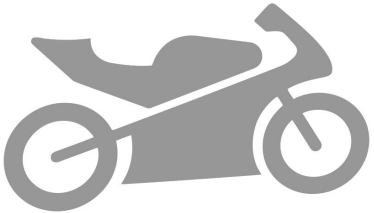
Pipelines from the beginning to the end in one job

Pipelines separated to stages and submitted as separate jobs

Production / Compute Nodes



- General computing nodes
- High memory nodes
- High CPU count nodes
- GPU nodes

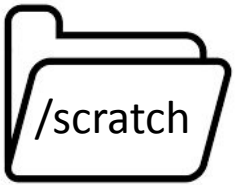


Different types of nodes accessible via **partitions** or **queues**

Filesystem

Often separate “home” and “scratch” storage.

- Small
- Backed-up
- Use for software and general scripts



- Large (1TB +)
- Not backed-up
- Use as “working directory” for processing data
- Make sure to regularly backup code from here

Use the HPC...

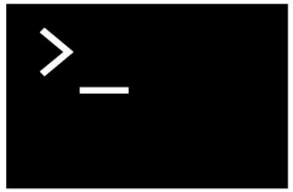
- ...**Ethically**
  - Do not use the login node for production runs
- ...**Smartly**
  - Optimise your jobs for CPU and time usage
  - Create universal and software-specific submission scripts (but never sample specific)
  - Reduce the number of CPU cores if it doesn't have a very significant effect to go in production earlier
  - Check production node usage
- ...**Efficiently**
  - Use free account (SL3) if the job is not urgent
  - Run multiple samples in parallel
  - Build up dependency chains

Exercise

- Take 5 minutes to answer this quiz
- Then we will discuss together

Connecting and working on a HPC

Working on Remote Servers: Overview



Unix Terminal

Login from the **terminal**

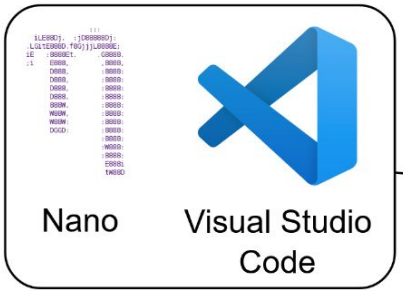
```
$ ssh participant@train.bio
```

Login nodes

submit scripts via the **job scheduler**

```
$ sbatch my_script.sh
```

Compute nodes



Nano

Visual Studio Code

Text Editor

Shared Filesystem

/home

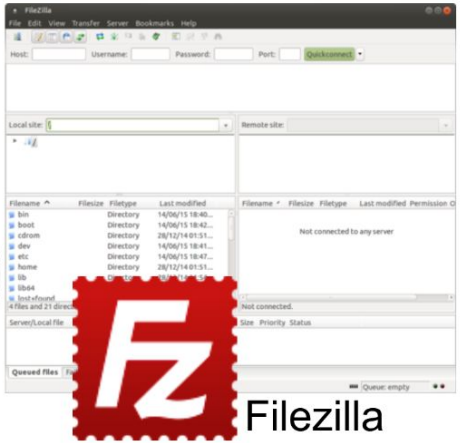
smaller, slower, backup

/scratch

larger, faster, no backup

Personal Computer

/Documents



Filezilla

Connecting to Remote Server: ssh

The ssh (secure shell) command is used to connect to a remote host

```
ssh  hm533@login.hpc.cam.ac.uk
```

username hostname

You will be asked for your password (and possibly two-factor authentication code, if setup in your HPC):

```
(hm533@login.hpc.cam.ac.uk) Password:
```

Your terminal should change to indicate you are on the remote server:

```
hm533@login-p-1.hpc.cam.ac.uk ~$
```

⚠️ As you type the password, nothing shows up! This is a safety mechanism, but your password is being captured as you type.

💡 Paste your password in:

Linux: **Ctrl + Shift + V**

macOS: ⌘ **+ V**

MobaXterm (Windows): **right-click**

Exercise

Login to the training HPC:

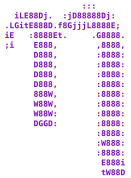
- Email from ‘`support@hpc.cam.ac.uk`’ contains:
  - Username
  - Link to get password (passcode will be given on the day)

- Use the `ssh` command to login into the hpc:
  - Hostname: `login.hpc.cam.ac.uk`.

- Your “scratch” folder is located in `~/rds/hpc-work`. Navigate to that folder and look at which files are present.

- Use the `who` command to see how many users are sharing this login node with you.

Editing Files: nano , VS Code, vim



nano

- Command-line text editor available on most Linux distributions
- **Ctrl + X** to exit nano
  - It will ask if you want to save the file → type **Y** (yes)
  - It will ask to confirm the file name → press **Enter**

VS Code

- GUI-based software
- Has the capability to connect to remote servers using the ssh protocol
  - Instructions in the course materials

vim

- Command-line text editor available on most Linux distributions
- Very advanced (but steep learning curve)
- If you know what this is, you don't need us to tell you about text editors

Job submission with SLURM

SLURM Job Scheduler

SLURM is the software that manages the job queue:

- It decides when each jobs runs, depending on the availability of compute nodes and the resources requested for that job (CPUs and memory).
- Your job is given an initial “queue position” that depends on:
  - Your account priority level (e.g. paid vs free tier)
  - Resources requested: the more CPUs/memory/time you ask for, the further back in the queue you start
- The “queue position” improves the longer the job spends in the queue

SLURM: Key Commands

- `sbatch` → submit a shell script to the queue
- `squeue` → see all the jobs in the queue
- `squeue -u USERNAME` → see your jobs only
- `scancel JOBID` → cancel the job with the specified ID
- `scancel -u USERNAME` → cancel all *your* jobs
- `seff JOBID` → get efficiency information about your job

Let's do a demo of all these!

SLURM: Summary

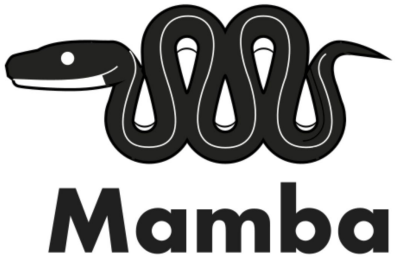
- Write your commands in a shell script
- Configure your job options using `#SBATCH` directives
- Use `$SLURM_*` environment variables to further customise your commands.
- Test your code by requesting a terminal on a compute node using `sintr` (interactive jobs)
  - all the options available with `sbatch` are also available to `sintr` (e.g. `-c`, `--mem`, `-t`, etc.)
  - Often `sintr` jobs are limited in maximum time they can run for (e.g. 1h at Cambridge), because this is not a very efficient way to run large-scale analyses.

Managing Software

Managing Software

Solution 1: pre-installed software

- Available through the ***Modules*** package (if the HPC admins set this up)



Solution 2: install it yourself locally

(remember you don't have admin permissions)

- Compile software from source → can be more challenging (we're not covering it here)
- Use a package manager → **Mamba** (previously Conda)

Managing Software: Modules

- List all the software packages available:

```
module avail
```

- Search for a particular package:

```
module avail -i --contains 'bowtie'
    bowtie2-2.3.1
    bowtie2-2.3.5
```

- Make the software available in my environment:

```
module load bowtie2-2.3.1
module load samtools-1.9
```

The *modules* package adds software to the user's $PATH

Remember to include the `module load` command in your SLURM submission script for each package you want to use

Managing Software: Modules

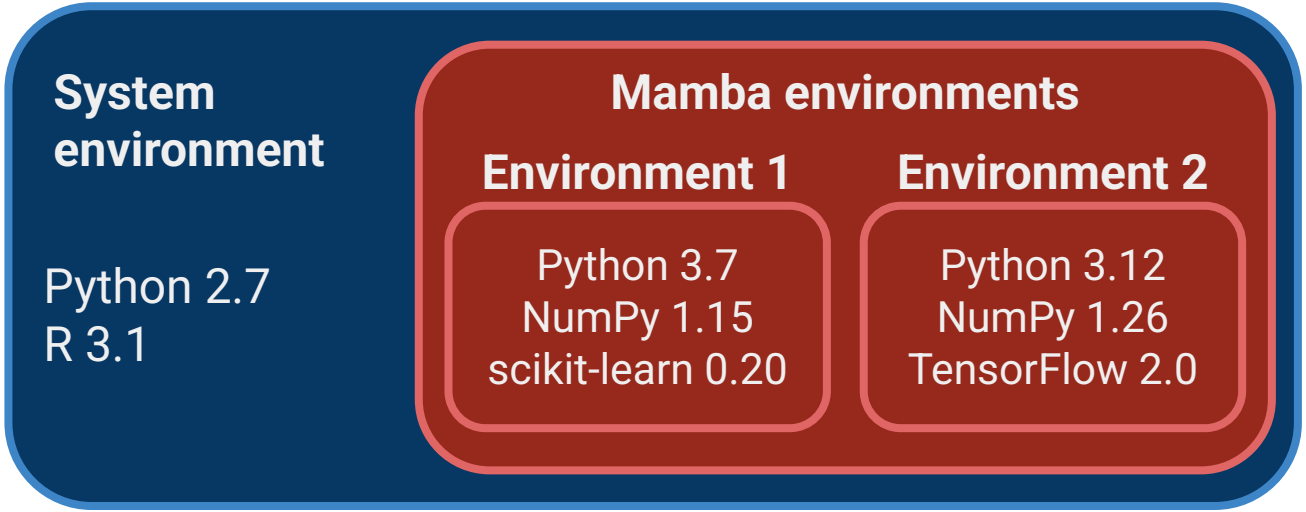
Pros:

- Ready and easy to use - no setup time for the user
- No need to worry about installation dependencies, etc - HPC admins take care of this for you (even if the software is kind of picky about dependencies).
- Software is compiled in an optimal way for the specific hardware of the HPC (may often increase speed and efficiency of the software)

Cons:

- If a software/version is not available, need to request it from the HPC admins, which may take some time.

Managing Software: Mamba/Conda

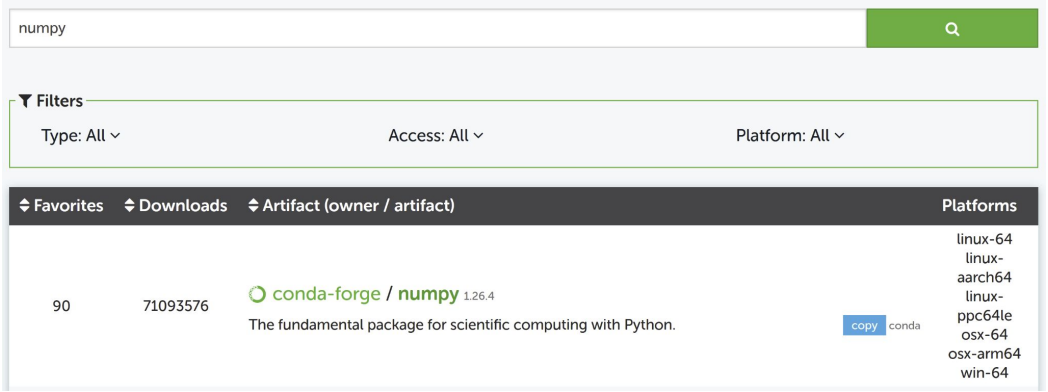


Mamba

CONDA

Managing Software: Conda

- Check which packages are available to install via *Mamba*: anaconda.org



- Create an environment:

```
mamba create -n datasci
```

- Install the package(s):

```
mamba install -n datasci -c conda-forge numpy=1.26.4
mamba install -n datasci -c conda-forge matplotlib=3.8.3
```

- To make all packages in the environment available:

```
mamba activate datasci
```

Managing Software: Conda

⚠️ Warning!

When submitting jobs to SLURM on the HPC you need to include the following lines of code:

```
# Always add these two commands to your scripts
eval "$(conda shell.bash hook)"
source $CONDA_PREFIX/etc/profile.d/mamba.sh

# then you can activate the environment
mamba activate datasci
```

Managing Software: Conda

Our Mamba installation instructions include a step to automatically search the two most popular channels/repositories:

- `conda-forge` for scientific computing software
- `bioconda` for bioinformatics software

Therefore, you can leave the channel out of the install command. These are all equivalent:

```
mamba install -n datasci -c conda-forge numpy=1.26.4
```

```
mamba install -n datasci conda-forge::numpy=1.26.4
```

```
mamba install -n datasci numpy=1.26.4
```

Managing Software: Conda

Pros:

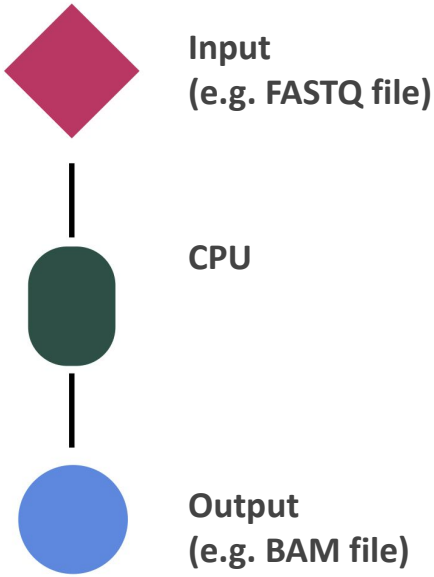
- Easier than compiling software yourself and automatically installs dependencies
- Installs software locally (by default in /home directory)
- Encapsulate different software versions in environments
- Can recreate an environment in another computer easily (see *Conda* docs)

Cons:

- Packages are not optimally compiled for the hardware in use
- For complex environments dependency conflicts can be hard (or impossible) to resolve
- Not every software is available through Mamba/Conda
- Can take a lot of space (tip: run `mamba clean` to remove unused and cached packages)

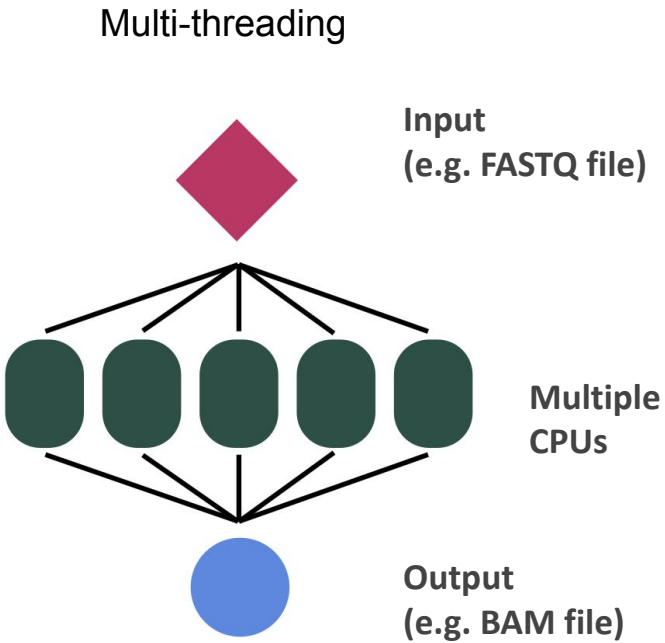
Job Arrays

Parallel Jobs: Speed Processing Time



- **Cores**, **threads** and **processors** are often used synonymously, usually they all indicate a CPU worker
  (although technically they are different things)
- Imagine that processing one input file takes 5 hours
- 100 files takes 500 hours (~ 21 days)

Parallel Jobs: Speed Processing Time



- Some software tools support multi-threading to speed things up
  (although speed gains aren't usually linear)
- Imagine that using 10 CPUs speeds things from 5 hours to 1 hour
- 100 files still takes 100 hours (~ 4 days) if we run them serially

Parallel Jobs in SLURM: ~~For Loops~~

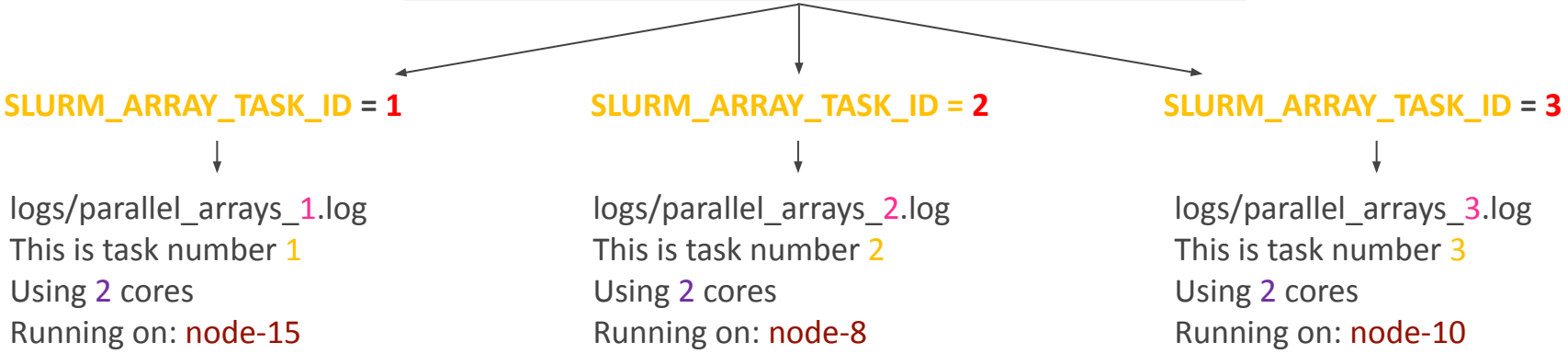
```
for eachFile in filesList
do
  sbatch jobScript.sh eachFile
done
```

- Job submission using loops is **not efficient** on the SLURM workload manager
- Not recommended

Parallel Jobs in SLURM: Job Arrays

```
#!/bin/bash
#SBATCH -D  /scratch/participant/hpc_workshop
#SBATCH -o  logs/parallel_arrays_%a.log
#SBATCH -c  2
#SBATCH --mem=1G
#SBATCH -a  1-3

echo "This is task number $SLURM_ARRAY_TASK_ID"
echo "Using $SLURM_CPUS_PER_TASK CPUs cores"
echo "Running on:"
hostname
```



Advantages and Limitations of Job Arrays

- Advantages
  - Job submission is quite fast: ~30,000 jobs/ 1-2 milliseconds
  - Faster than using “for loops”
  - Job management is easy for us and to SLURM
  - Job array can be handled as a whole
  - Individual jobs in an array can be handled independently

Put it in practice:
Exercise

- Limitations
  - Each job in the array will request the same resources, like CPUs, memory, time etc.

Job Arrays: `$SLURM_ARRAY_TASK_ID` Tricks

For stochastic simulations can use it to set a seed (for reproducibility)

```
#SBATCH -a 1-100

python    my_simulation_script.py    $SLURM_ARRAY_TASK_ID
```

Write your script so that it takes a number as input, which is then used to set a seed for a random number generator

(You may not always want to do this - if you're running many simulations in your work, maybe you shouldn't use the same set of seeds all the time)

Job Arrays: `$SLURM_ARRAY_TASK_ID` Tricks

Different inputs

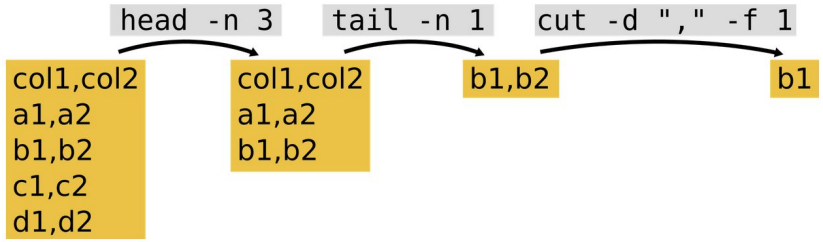
1. Prepare an input sample sheet (e.g. CSV format)

```
sample,input
patient1,data/XYZ10231.fq
patient2,data/XYZ19381.fq
```

Job Arrays: `$SLURM_ARRAY_TASK_ID` Tricks

Different inputs

1. Prepare an input sample sheet (e.g. CSV format)
2. Unix commands to get nth line of a file:



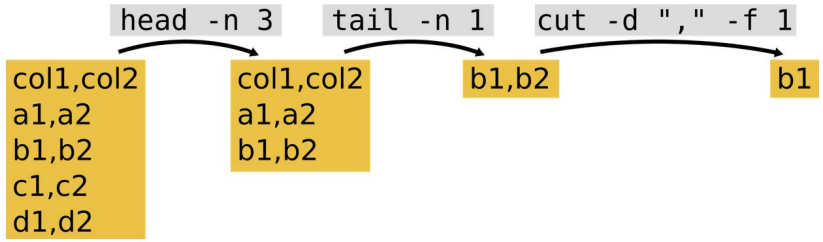

```
sample,input
patient1,data/XYZ10231.fq
patient2,data/XYZ19381.fq
```

Job Arrays: `$SLURM_ARRAY_TASK_ID` Tricks

Different inputs

1. Prepare an input sample sheet (e.g. CSV format)
2. Use some Unix command line skills

```
sample,input
patient1,data/XYZ10231.fq
patient2,data/XYZ19381.fq
```



Do Exercise
Green sticky → finished
Red sticky → help!

```
#SBATCH -a 2-3

SAMPLE=$(head -n $SLURM_ARRAY_TASK_ID samplesheet.csv | tail -n 1 | cut -d "," -f 1)
 INPUT=$(head -n $SLURM_ARRAY_TASK_ID samplesheet.csv | tail -n 1 | cut -d "," -f 2)

command --input ${INPUT} --output results/${SAMPLE}.out
```

Exercise

Using job arrays with multiple inputs

Two equivalent versions of the exercise:

- Bioinformatics example - mapping sequencing reads to a reference genome for several input samples
- Simulation example - exploring turing patterns varying the model parameters

Job Dependencies

Job Dependencies: Syntax

Linear pipeline where each script depends on the output of a previous script:

job1.sh → job2.sh → job3.sh

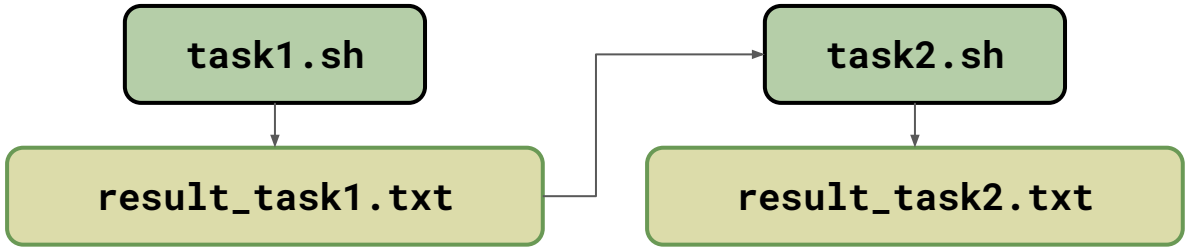
```
$ sbatch job1.sh
Submitted batch job 349

$ sbatch --dependency=afterok:349 job2.sh
Submitted batch job 350

$ sbatch --dependency=afterok:350 job3.sh
Submitted batch job 351
```

Job Dependencies: “afterok”

Useful for pipelines with a “linear” chain of job dependencies



- Task1 creates a file
- Task2 takes this file to produce the second file
- Task2 should only run if task1 is successful

Demo in `hpc_workshop/dependency/ok`

Job Dependencies: Capturing Job ID

```
# first task of our pipeline
# capture JOBID into a variable
run1_id=$(sbatch --parsable task1.sh)

# second task of our pipeline
# use the previous variable here
sbatch --dependency afterok:${run1_id} task2.sh
```

You can write a long linear pipeline like this in a shell script and run as:
`bash submit_pipeline.sh`

(note: this is not submitted to SLURM → the script is doing the submission for us instead)

Types of Job Dependencies

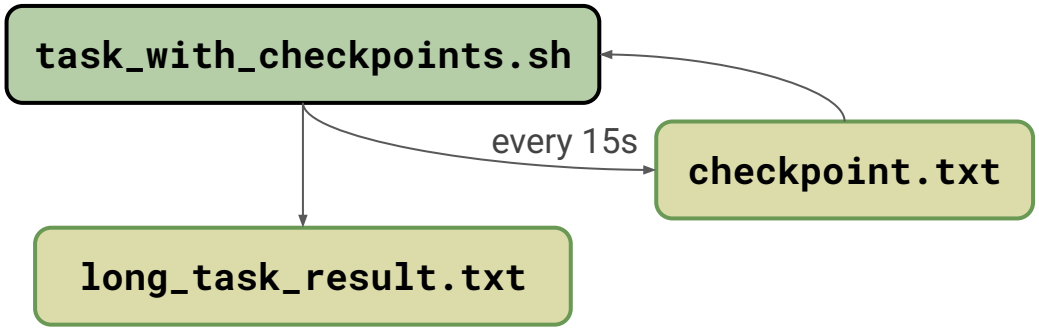
afterok:jobid[:jobid...]	job can begin after the specified jobs **have completed with an exit code of zero**
afternotok:jobid[:jobid...]	job can begin after the specified jobs **have failed**
singleton	jobs can begin execution after **all previous jobs with the same name and user have ended**. This is useful to collate results of a swarm or to send a notification at the end of a swarm.
after:jobid[:jobid...]	job can begin after the specified jobs **have started**
afterany:jobid[:jobid...]	job can begin after the specified jobs **have terminated**

Job Dependencies: “afternotok”

- Alternative rescue pathway in a pipeline
- Checkpoint and restart
  - Enable jobs to run longer than time limit
  - Improve jobs’ throughput by exploiting the holes in the SLURM schedule
  - Debug long-running jobs by pausing just before the error & restarting from that point multiple times

- Time limit is an everyday problem at the Cambridge University HPC (max 12h/36h on SL3/SL2)

Job Dependencies: “afternotok”

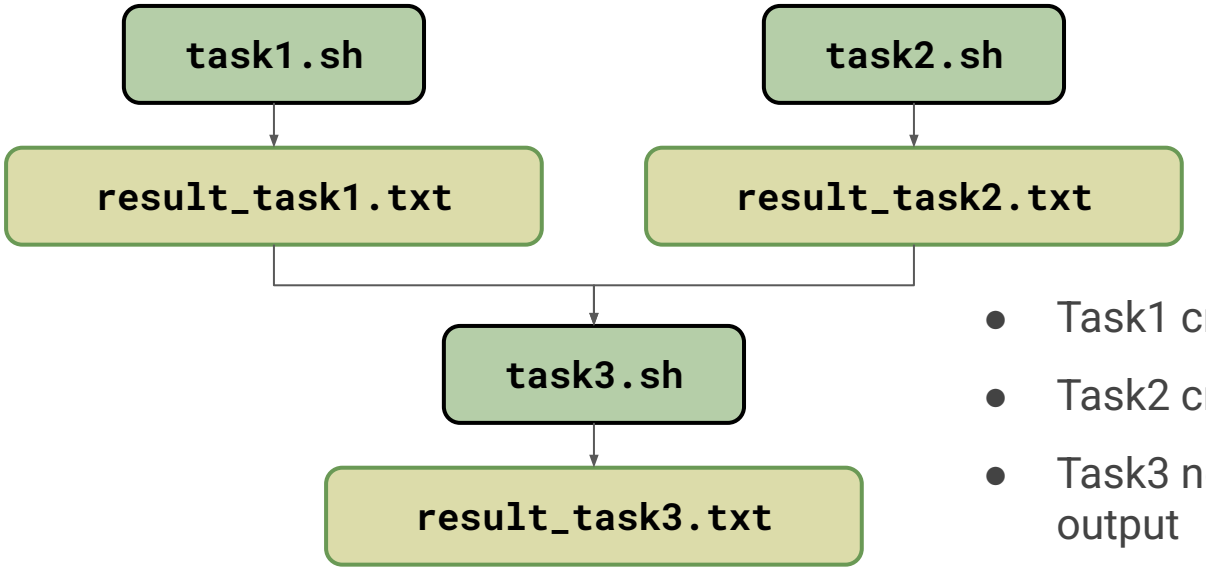


- Counts to 10 increasing by 1 every 15 secs
- At each step it saves the current number in a file `checkpoint.txt`
- If `checkpoint.txt` exists the task resumes from that point
- We will submit our job with 1 minute only → meaning we need 3 reruns to complete

Demo in **`hpc_workshop/dependency/notok`**

Job Dependencies: “singleton”

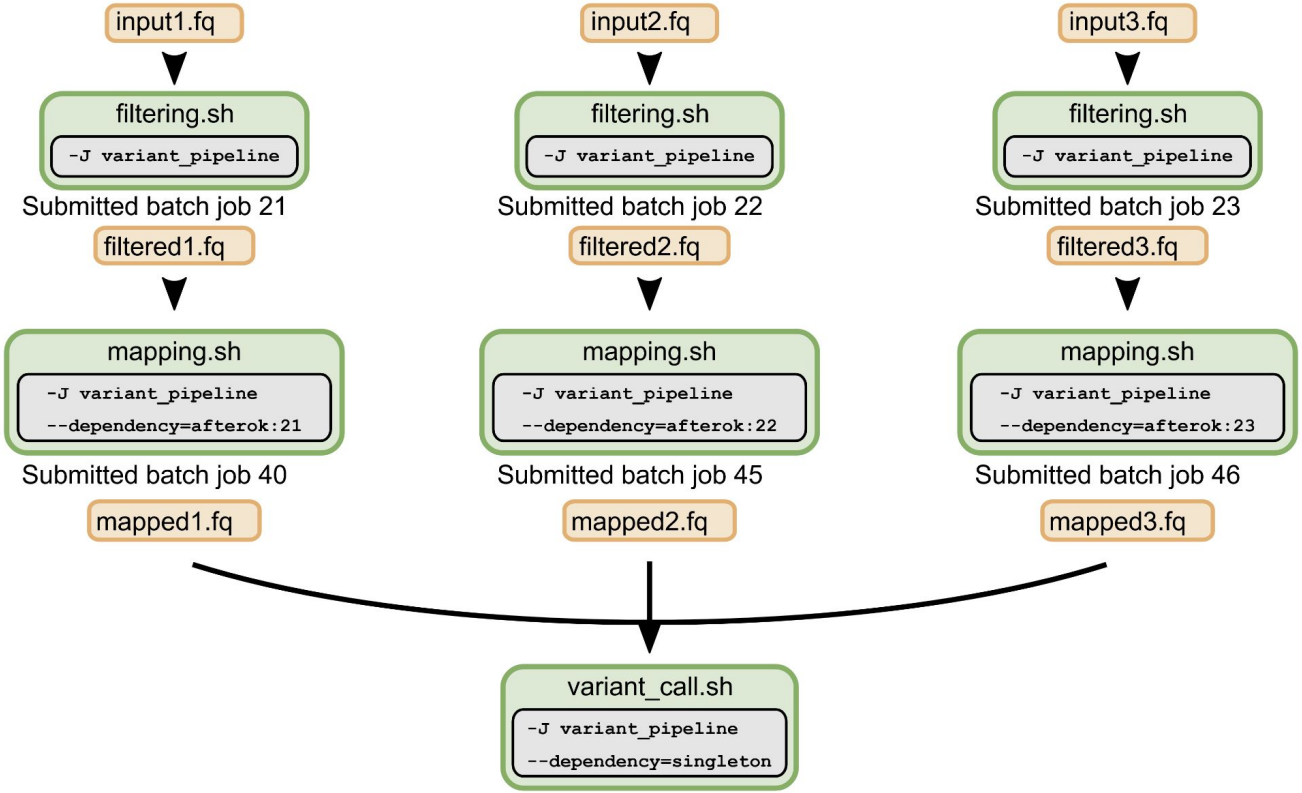
Useful for tasks with multiple dependencies



- Task1 creates a file
- Task2 creates another file
- Task3 needs both to produce a third output

Demo in `hpc_workshop/dependency/singleton`

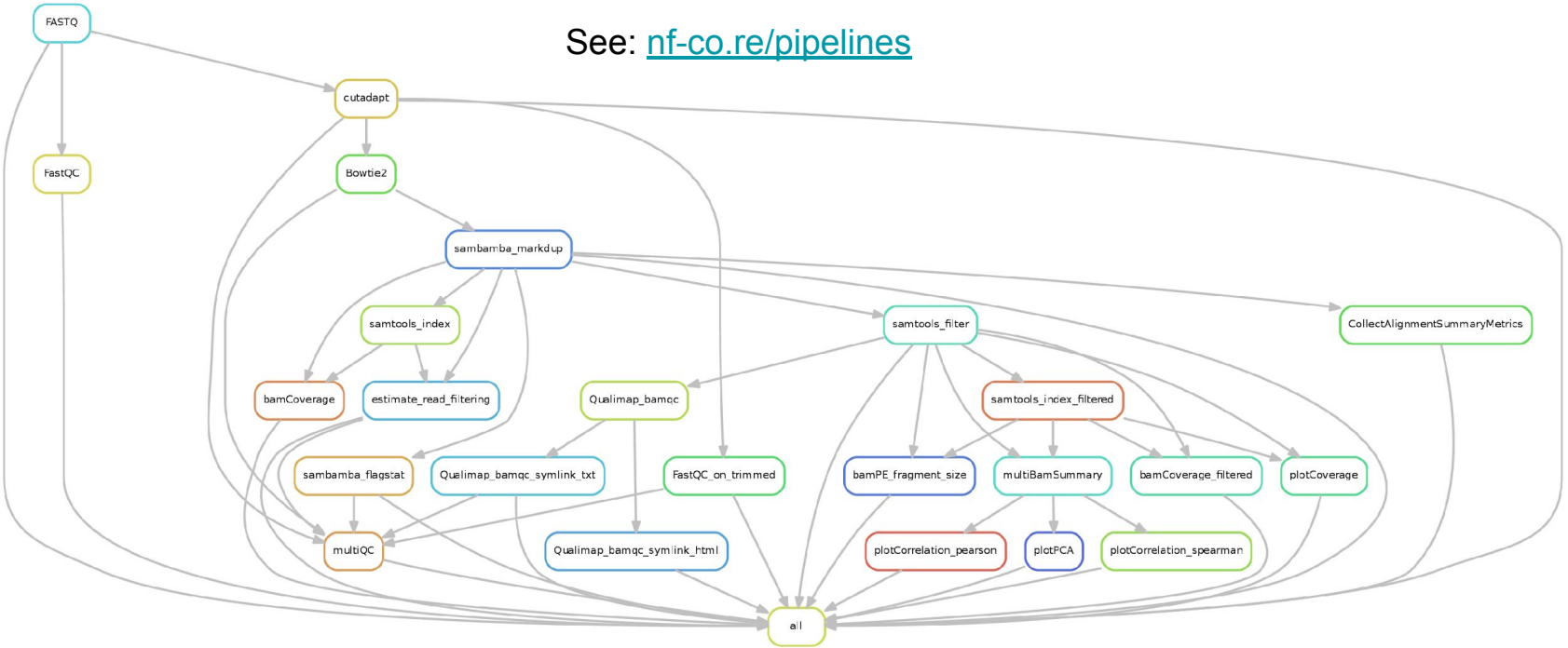
Job Dependencies: Complex Pipelines



Job Dependencies: Complex Pipelines

For complex pipelines using dedicated workflow-management software such as ***Snakemake*** or ***Nextflow*** may be more suitable

See: nf-co.re/pipelines

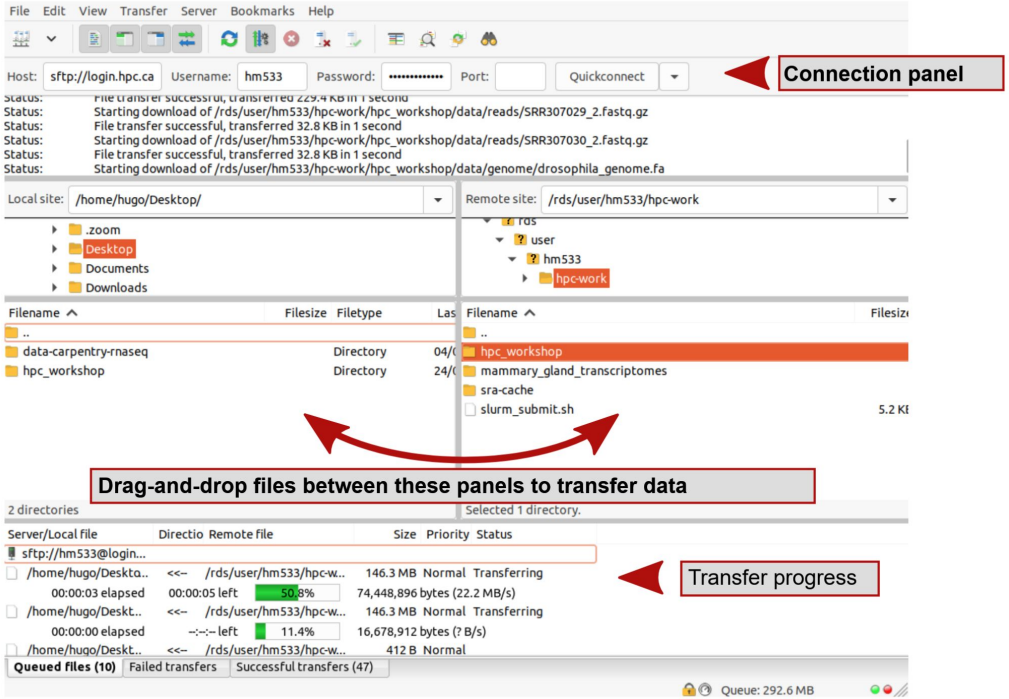


Moving Files

Moving Files to Remote Servers

	Filezilla	SCP	Rsync
Interface	GUI	Command Line	Command Line
Data synchronisation	yes	no	yes

Moving Files: *FileZilla*



Pros:

- Easy to use (GUI-based)
- Data synchronization
- Can pause/restart transfer if needed

Cons:

- Not suitable if you want to transfer data between two remote servers (e.g. two HPC servers)

Moving Files: scp

My computer (e.g. a macOS):

```
/Users
  |_robin
     |_Documents
        |_awesome_proj
           |_ data
```

The HPC filesystem:

```
/home
  |_rob123
     |_scratch
        |_ awesome_proj
           |_ data
           |_ results
```

I want to transfer the data folder

```
$ cd ~/Documents
$ scp  -r  awesome_proj/data  rob123@login.hpc:scratch/awesome_proj
```

- `scp` — copy directories recursively (like `cp`)
- `-r` — copy directories recursively (like `cp`)
- `awesome_proj/data` — The **source** directory/file I want to copy (relative to my current dir)
- `rob123@login.hpc:` — The credentials to access the HPC (same as with `ssh`)
- `:` — A separator
- `scratch/awesome_proj` — The **destination** I want to copy it into (relative to `/home/rob123`)

Moving Files: scp

My computer (e.g. a macOS):

```
/Users
  |_robin
     |_Documents
        |_awesome_proj
           |_ data
```

I want to transfer the results folder

The HPC filesystem:

```
/home
  |_rob123
     |_scratch
        |_ awesome_proj
           |_ data
           |_ results
```

```
$ cd ~/Documents
$ scp  -r  rob123@login.hpc:scratch/awesome_proj/results  awesome_proj
```

- `scp` — copy directories recursively (like `cp`)
- `-r` — copy directories recursively (like `cp`)
- `rob123@login.hpc` — The credentials to access the HPC (same as with `ssh`)
- `:` — A separator
- `scratch/awesome_proj/results` — The **source** directory/file I want to copy (relative to `/home/rob123`)
- `awesome_proj` — The **destination** I want to copy it into (relative to my current dir)

Moving Files: scp

Pros:

- Works from the command line
- Works similarly to standard `cp` command
- Can be used to move data between two remote servers:
  - First login to one of the servers with `ssh`
  - Then use `scp` from that server to the other (remote) server

Cons:

- Always copies all the files and overwrites them if they already exist (no sync ability)

Moving Files: rsync

My computer (e.g. a macOS):

```
/Users
  |_robin
     |_Documents
        |_awesome_proj
           |_ data
```

The HPC filesystem:

```
/home
  |_rob123
     |_scratch
        |_ awesome_proj
           |_ data
           |_ results
```

I want to sync the data folder

```
$ cd ~/Documents
$ rsync  -avhu awesome_proj/data/  rob123@login.hpc:scratch/awesome_proj/data/
```

Only transfer new files (more explanation in following slides)

The **source** directory/file I want to synch (relative to my current dir)

The credentials to access the HPC (same as with `ssh`)

A separator

The **destination** I want to synch it into (relative to `/home/rob123`)

Moving Files: rsync

My computer (e.g. a macOS):

```
/Users
  |_robin
     |_Documents
        |_awesome_proj
           |_ data
```

The HPC filesystem:

```
/home
  |_rob123
     |_scratch
        |_ awesome_proj
           |_ data
           |_ results
```

I want to sync the data folder

```
$ cd ~/Documents
$ rsync  -avhu awesome_proj/data/  rob123@login.hpc:scratch/awesome_proj/data/
```

⚠️ ⚠️ ⚠️

The / in the source path is important!

- If you include / at the end → transfer the **contents inside** the folder to the destination
- If you don't → transfer the actual **entire folder** to the destination

Moving Files: rsync

My computer (e.g. a macOS):

```
/Users
  |_robin
     |_Documents
        |_awesome_proj
           |_ data
```

The HPC filesystem:

```
/home
  |_rob123
     |_scratch
        |_ awesome_proj
           |_ data
              |_ data
           |_ results
```

This would happen if you didn't include the / in source path

```
rsync  -avhu awesome_proj/data rob123@login.hpc:scratch/awesome_proj/data/
```

Moving Files: rsync

My computer (e.g. a macOS):

```
/Users
  |_robin
     |_Documents
        |_awesome_proj
           |_ data
```

The HPC filesystem:

```
/home
  |_rob123
     |_scratch
        |_ awesome_proj
           |_ data
           |_ results
```

I want to transfer the results folder

```
$ cd ~/Documents
$ rsync  -avhu  rob123@login.hpc:scratch/awesome_proj/results  awesome_proj/
```

Only transfer new files (more explanation in following slides)

The credentials to access the HPC (same as with `ssh`)

A separator

The **source** directory/file I want to copy (relative to `/home/rob123`)

The **destination** I want to copy it into (relative to my current dir)

Moving Files: rsync

My computer (e.g. a macOS):

```
/Users
  |_robin
     |_Documents
        |_awesome_proj
           |_ data
           |_ results
```

The HPC filesystem:

```
/home
  |_rob123
     |_scratch
        |_ awesome_proj
           |_ data
           |_ results
```


```
$ cd ~/Documents
$ rsync  -avhu  rob123@login.hpc:scratch/awesome_proj/results/ awesome_proj/
```

⚠️ ⚠️ ⚠️

The **/ at the end of the source path** is important!

By excluding the / we are saying: “transfer the **entire** *results* folder to *awesome_proj*”

Moving Files: rsync

My computer (e.g. a macOS):

```
/Users
  |_robin
     |_Documents
        |_awesome_proj
           |_ data
           |_ file1.csv
           |_ file2.txt
```

The HPC filesystem:

```
/home
  |_rob123
     |_scratch
        |_ awesome_proj
           |_ data
           |_ results
              |_ file1.csv
              |_ file2.txt
```

This would happen if you included the / in the source path

```
rsync  -avhu  rob123@login.hpc:scratch/awesome_proj/results/  awesome_proj/
```

With / we are saying: “transfer the **contents** of *results* to *awesome_proj*”

Moving Files: `rsync`

Options:

- `-a` → only transfer files that have changed and preserve their timestamps and other properties
- `-u` → in addition, only transfer files **if they are newer** compared to the destination (avoids overwriting newer files with older versions by accident)
- `-h` → print file sizes in human format like Mb, Gb, etc.
- `-v` → verbose mode, print some information about what was transferred
- `--progress` → show the progress of file transfer (useful when transferring larger files
- `-z` → compress the data in transit (can save some bandwidth)

Moving Files: `rsync`

💡 Tip:

`--dry-run` option → shows you what files/folders `rsync` will transfer with your command, but not actually do the transfer.

Great way to make sure you specified your paths and options correctly!

Moving Files: `rsync`

Pros:

- Works from the command line
- Can be used to move data between two remote servers
- Much more flexible than `scp` → can synchronize files saving time and bandwidth
- The option `--dry-run` allows testing what the command would do before actually running

Cons:

- Many more options can make it a steeper learning curve
- The subtle `/` at the end of the source path can cause confusion

Exercise

- Using your choice of *rsync* or *scp*, copy the files you've been working on during this workshop to your own computer.

- If you want you can also try to move some files from your computer to the HPC.